

Functional Programming Basics

What is functional programming

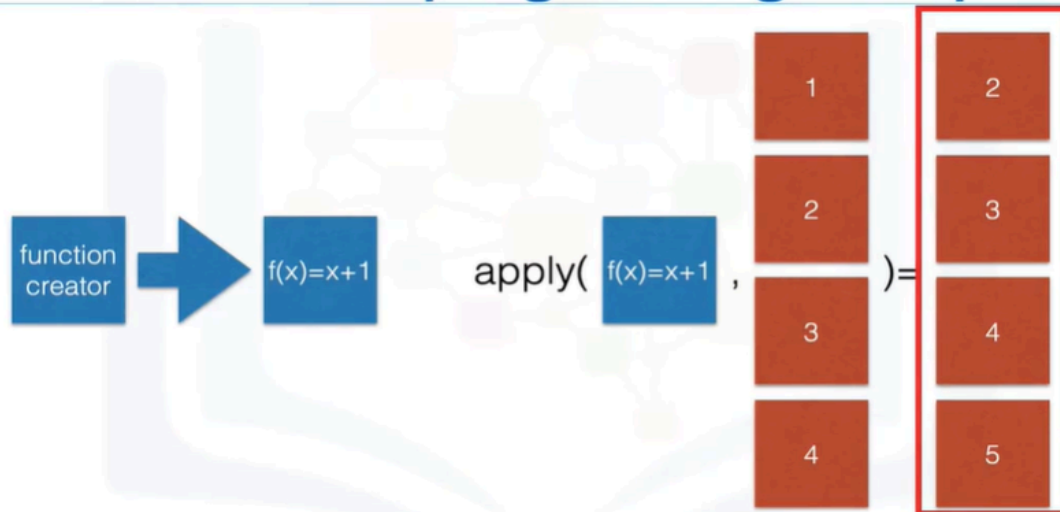
- A mathematical “function” programming style
- Follows a declarative programming model
- Emphasizes “What” instead of “How-to”
- Uses “expressions” instead of “statements”

- So, what is Functional Programming? Functional Programming, or FP, is a style of programming that follows the mathematical function format. Think of an algebra class with the f of x notation. The notation is declarative in nature as opposed to being imperative. By declarative, we mean that the emphasis of the code or program is on the “what” of the solution as opposed to the “how to” of the solution. Declarative syntax abstracts out the implementation details and only emphasizes on the final output, restated, “the what.” We use expressions in functional programming, like the expression f of x as mentioned earlier.

What is functional programming

- First implementation was LISt Programming Language (LISP) in the 1950s
 - Scala most recent representative (Python, R and Java also provide rudimentary support for functional programming)
 - Functions are first class citizens in Scala
-
- Historically, LISt Processing Language, known as LISP, was the first functional programming language, starting in the 1950s. But today there are many functional programming language options including Scala, Python, R, and Java. Scala is the most recent representative in this family of programming languages. Apache Spark is written mainly in Scala, which treats functions as first-class citizens. Functions in Scala can be passed as arguments to other functions, returned by other functions, and used as variables.

A functional programming example



- Here is a simple example of a functional program that increments a number by one. We define the function f of x equal to $x + 1$. We can then apply this function to a list of size four as shown in the figure, and the program increments every element in the list by one. That's the advantage of using functional programming. You can directly apply functions to an entire list or array!

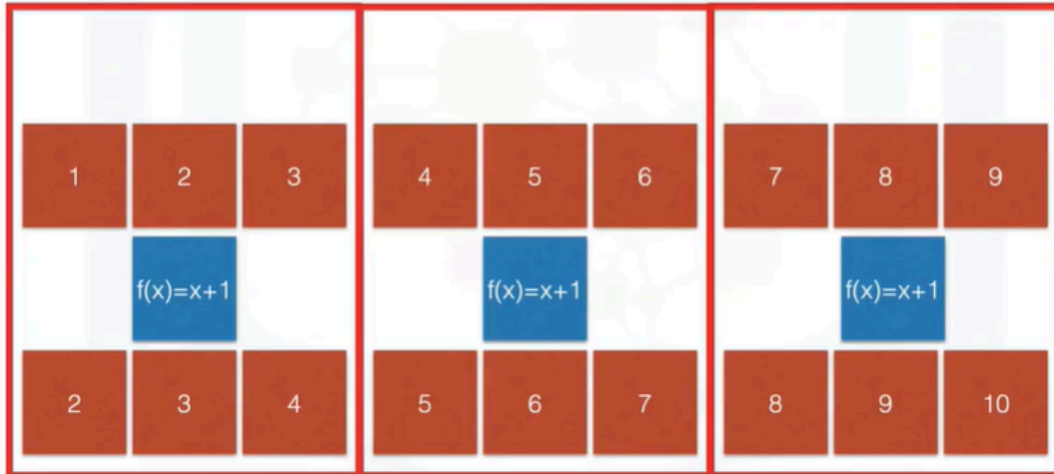
A traditional procedural example

A more traditional procedural way to achieve the same result:

```
// A Python function to add 1 to each element of a List
def increment(myList):
    N = size(myList)
    for i in range(N): #go over each element
        myList[i] += 1 #increment each element
    return myList
```

- If you perform the same task in an imperative paradigm using procedural code, you would construct a "for-loop" that iterates over the array and increment each element by 1. This programming paradigm differs from the preceding example by explicitly listing all the steps to be performed, including the "go over" for each element, incrementing each element by one. This code places emphasis on the "how-to." Compare and contrast this traditional programming example to the functional programming example that emphasized on the "what" part of the solution.

Parallelization



- Parallelization is one of the main benefits of Functional Programming. Consider the previous example where the program incremented by one up to 4. Here is a larger array, from one to nine. You can split the task into three different computing chunks, which can be referred to as nodes, click one, and run those three tasks or nodes in parallel. This method's beauty is that you do not need to change any of the function definitions or the code. All you need to do is to run more than one instance of the task in parallel. By applying implicit parallelization functional programming capabilities, you can scale the algorithm to any size by adding more compute and resources without modifying the program or code. The result is the same as if the function exists on only a single node. However, in this case, the function ran three times in parallel without requiring any changes to the function or code.

What are lambda functions?

- Lambda calculus functions, or operators, are anonymous functions that enable functional programming

Scala

```
// an example lambda  
operator in Scala to add  
2 numbers  
val add = (x:Int, y:Int)  
=> x + y  
println(add(1,2))
```

Python

```
// an example lambda  
function in Python to add  
2 numbers  
add = lambda x, y : x + y  
print(add(1,2))
```

- Functional programming applies a mathematical concept called lambda calculus. To keep the explanation simple, lambda calculus basically says that every computation can be expressed as an anonymous function which is applied to a data set. Lambda functions or operators are anonymous functions used to write functional programming code. Here, we have the code written in functional programming style using lambda functions and operators to add two numbers using two popular languages, Scala as seen on the left, and Python, as seen on the right. Note how both codes are similar in flow, and abstract out the result directly, a hallmark of the declarative paradigm of programming.

Lambda functions and Spark

- Spark parallelizes computations using the lambda calculus
- All functional Spark programs are inherently parallelizable



- Apache Spark is capable of quickly working through big data, by lambda functions distributing work between worker nodes and parallelized computations. All Spark programs implemented this way are inherently parallel, so it doesn't matter if you analyze one kilobyte or one petabyte of data. Just add additional resources to the Spark cluster and you're done.